Predictive Maintenance of CNC Machines Using Integrated Risk Assessment and Deep Learning Forecasting

Huseyin Ali Sarikaya,

Mudanya University. Faculty of Engineering, Architecture and Design. Department of Industrial Engineering. 16940, Mudanya, Bursa, Turkey.

Abstract

Small and medium-sized enterprises (SMEs) often lag behind larger corporations in adopting digital transformation, particularly in maintenance operations. Reactive or fixed-schedule maintenance practices frequently lead to costly downtime and inefficiencies. This study introduces a hybrid predictive maintenance framework tailored for CNC machining centers, integrating quantitative risk assessment with deep learning-based time-series forecasting. Maintenance data collected over two years (2022–2024) from an automotive components manufacturer operating 17 CNC machines were analyzed to calculate the Mean Time to Failure (MTTF) for 13 fault categories.

A dual-stage risk prioritization was applied: initially using the Fine–Kinney method to quantify risk levels based on probability, frequency, and severity, followed by a multi-criteria decision-making (MCDM) ranking via the TOPSIS method using detectability, repair cost, and MTTF as evaluation criteria. Coolant system failures, table malfunctions, and automated tool changer (ATC) errors were identified as the most critical failure types.

For each selected failure type, time-series forecasting models were developed using three recurrent neural network (RNN) architectures: a classical RNN implemented in MATLAB and more advanced LSTM and GRU models implemented in Python using TensorFlow/Keras. Each model was trained on normalized sequences of past failure intervals using a 70%-15%-15% data split for training, validation, and testing, respectively. Performance was assessed using MSE, RMSE, MAE, and Pearson's correlation coefficient (R). LSTM achieved the highest accuracy (e.g., coolant failure MSE ≈ 50 days², R ≈ 0.999), with GRU performing comparably and outperforming the standard RNN.

The proposed framework demonstrates that integrating risk analysis with predictive modeling enables SMEs to transition from reactive to proactive maintenance strategies, effectively minimizing unplanned downtime and enhancing overall equipment efficiency.

Keywords: Predictive Maintenance; Recurrent Neural Networks; LSTM; Fine-Kinney Risk Assessment; Time-Series Forecasting.

Corresponding author: huseyin.sarikaya@mudanya.edu.tr

1. Introduction

Maintenance costs account for a significant amount of production expenses (60-70%), and unexpected equipment breakdowns might completely halt production. Traditional planned maintenance on regular schedules frequently fails to prevent problems and can be inefficient. In response, predictive maintenance solutions have arisen, which use condition monitoring and data-driven models to predict failures and service equipment on-demand. Predictive maintenance can save around 30% of maintenance costs and prevent up to 75% of breakdowns (Matyas et al., 2017). However, many small

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

and medium-sized businesses (SMEs) have challenges in implementing such initiatives due to insufficient digitization and unstandardized data collecting.

Recurrent Neural Networks (RNNs) have proven particularly useful in processing time-series data in maintenance applications. Unlike feed-forward networks, RNNs maintain historical data via feedback connections, making them ideal for modeling sequences of failure events.

RNNs, first conceptualized by Werbos (1988), introduced the concept of training recurrent systems via backpropagation through time. Subsequently, Elman (1990) proposed a simple but effective architecture, known as the Elman network, that enables sequence modeling via context layers. However, standard RNNs suffer from vanishing gradient problems, limiting their ability to capture long-term dependencies. To overcome these limitations, Hochreiter and Schmidhuber (1997) introduced Long Short-Term Memory (LSTM) networks, which incorporate memory cells and gating mechanisms to preserve long-term information. Later, Cho et al. (2014) developed Gated Recurrent Units (GRUs) as a simplified and computationally efficient alternative to LSTM models. These networks have shown improved performance in a variety of domains, including battery life prediction (Zhang et al., 2018), rotor system monitoring (Jamal et al., 2020), and compressor maintenance (Harkat et al., 2023). The interpretability of LSTM-based maintenance models has also been investigated (for example, using Layer-wise Relevance Propagation) to increase prediction trust (Gupta et al., 2024).

Several notable studies have demonstrated the effectiveness of RNNs in fault prediction across various domains. Zhang et al. (2018) utilized LSTM-RNNs to predict the remaining useful life of lithium-ion batteries, significantly enhancing reliability. Jamal et al. (2020) advocated for the use of LSTM-RNNs in predictive maintenance, showing superior performance compared to traditional neural networks. Killeen and Patrick (2020) integrated IoT-based architecture with dynamic sensor selection, leading to improved fault prediction accuracy. Wu et al. (2021) enhanced the interpretability of predictive maintenance systems by employing layer-wise relevance propagation for bearing health monitoring. Zainuddin et al. (2021) achieved an 87% accuracy rate in predicting machine failures in the oil and gas sector using RNN-GRU models. Ruslan and Hassan (2022) demonstrated that incorporating crest factor and kurtosis metrics improved RNN performance in machinery fault prediction. Adamou et al. (2023) combined RNNs with digital twin technology and genetic algorithms, showcasing energy-efficient predictive maintenance for rotors. Harkat et al. (2023) implemented LSTM models in predictive maintenance workflows, increasing uptime for centrifugal compressors in Industry 4.0 environments. Finally, Dintén and Zorrilla (2024) developed a hybrid Transformer-GRU model integrated with Explainable AI (XAI), achieving optimal failure predictions using IoT-based datasets. These studies collectively highlight the versatility and advancements of RNN-based approaches in fault prediction and predictive maintenance.

In addition, risk analysis is critical for effective maintenance planning. By identifying and prioritizing significant failure modes, maintenance efforts can be directed to where they will be most useful. The Fine–Kinney method (Fine, 1971) systematically quantifies risk by evaluating probability, frequency, and severity. To improve accuracy under uncertainty, Fine-Kinney is frequently used in conjunction with multi-criteria decision approaches (AHP, fuzzy logic, TOPSIS, and so on). In multi-criteria decision-making contexts, the Technique for Order Preference by Similarity to Ideal Solution (TOPSIS) developed by Hwang and Yoon (1981) provides a structured framework to rank alternatives based on their closeness to the ideal solution. Recent research has used hybrid risk models in a variety of industries (energy, manufacturing, and safety) to prioritize maintenance operations. Organizations may ensure that the most harmful or frequent failures are addressed and managed proactively by combining risk assessment and predictive modelling.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

This study presents a hybrid approach to predictive maintenance in the CNC manufacturing context, which combines Fine-Kinney risk prioritization with RNN-based time-series prediction. In a case study conducted at a CNC machining factory in Konya (Turkey), we classified and analyzed two years of failure data to identify crucial failure categories. We then created and compared three RNN-type models - a conventional RNN, an LSTM network, and a GRU network - to forecast the time to next failure for the most important failure types. To ensure the solution's durability, models were constructed in two environments: MATLAB (with its Neural Network Toolbox for a nonlinear autoregressive network and Levenberg-Marquardt training) and Python (with TensorFlow/Keras for LSTM and GRU deep learning). Multiple metrics (MSE, RMSE, MAE, and correlation R) were used to evaluate the models' predictive performance, which was then represented using prediction plots and training learning curves. We show that, even with limited failure event data (dozens of incidents per category), RNN-based models can learn temporal trends and effectively forecast future failures, allowing maintenance teams to take proactive measures.

While many studies have used machine learning models or risk prioritization frameworks separately in the maintenance domain, this study is the first to combine the two into a single predictive maintenance framework. Specifically, the combination of Fine-Kinney risk scoring and TOPSIS-based multi-criteria ranking provides a complete and sophisticated approach to fault prioritization — a characteristic that has received little attention in existing literature. Furthermore, this study compares multiple recurrent neural network architectures, including a traditional RNN implemented in MATLAB (trained with Levenberg-Marquardt) and advanced LSTM and GRU networks implemented in Python/TensorFlow (trained with Adam optimizer), to demonstrate cross-platform applicability and robustness.

This study bridges the gap between theoretical research and industry application by including real-world failure records from an operational CNC manufacturing environment and customizing the predictive models to SMEs, who frequently lack broad sensor networks or huge datasets. As a result, it contributes greatly to the development of accessible, data-driven predictive maintenance techniques in the manufacturing sector.

In the following sections, a comprehensive methodological framework will be presented, detailing the data collection process, analytical techniques, and model development strategies employed in this study. Following the methodology, the results obtained from the analyses will be systematically reported and interpreted. Finally, the paper will conclude with a discussion of key findings, their practical implications, and suggestions for future research directions.

2. Methodology

2.1 Case Study: Data Collection and Failure Classification

The research was carried out at an automotive parts manufacturing company in Konya that operated 17 CNC machines. Maintenance logs were gathered during a two-year period (2022-2024), containing the date of each failure incidence and a brief description. Initially, the failure data was unstructured, making analysis difficult. We used a failure classification technique to bring together similar failure instances. All reported problems were divided into 13 key areas based on expert consultation and maintenance record review: coolant system, switch, spindle, conveyor, hydraulic, screw, ruler, axis, APC (Automatic Pallet Changer), air pressure, slide, ATC (Automatic Tool Changer), and table. This segmentation made targeted analysis easier because each failure category had unique causes and maintenance practices.

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

Then, we conducted a failure occurrence analysis to identify the frequency of each failure category. Table 1 highlights the failure ratio by reason. The cooling system, ATC, table, and conveyor were the most common failures, accounting for 16-17% of total failures. Switches, rulers, APCs, and air pressure have modest occurrence rates (~2%). These frequency ratios indicate which failure modes are frequent and thus viable candidates for predictive modeling.

Table 1. Ratio of individual failure causes to total number of failures

Failure Type	Occurrences	Occurrence Ratio
Coolant	50	0.1701
Switch	7	0.0238
Spindle	21	0.0714
Conveyor	47	0.1599
Hydraulic	21	0.0714
Screw	9	0.0306
Ruler	6	0.0204
Axis	12	0.0408
APC	6	0.0204
Air Pressure	7	0.0238
Slide	14	0.0476
ATC	46	0.1565
Table	48	0.1633

Mean Time to Failure (MTTF) Calculation

MTTF is a fundamental reliability metric that estimates the expected lifespan of non-repairable components. It is frequently used in predictive maintenance and asset management (Ebeling 2019). The MTTF for each failure type was obtained by averaging the intervals between consecutive failure occurrences, as stated in Equation (1) for timestamps: $t_1 < t_2 < \cdots < t_N$

$$MTTF = \frac{1}{N-1}\sum_{i=1}^{N-1}(t_{i+1} - t_i) \tag{1}$$

We also normalized these figures to evaluate relative failure rates by calculating the ratio of each MTTF to the total. Table 2 presents a summary of the MTTF outcomes. Air-pressure system failures had the longest MTTF (~148 days) due to their rarity but severity, while coolant system failures had a shorter MTTF (~17.6 days) due to their high frequency. These calculations measure the frequency with which each fault occurs, forming the foundation for risk grading.

Table 2. MTTF and corresponding ratios

Failure Type	MTTF	Ratio
Coolant	17.60	0.03
Switch	54.75	0.09
Spindle	26.75	0.04
Conveyor	18.06	0.03

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

Hydraulic	39.31	0.06
Screw	73.20	0.12
Ruler	41.33	0.07
Axis	50.67	0.08
APC	68.75	0.11
Air Pressure	147.75	0.24
Slide	50.30	0.08
ATC	12.94	0.02
Table	17.66	0.03
Total	619.06	1.00

To ensure a robust and comprehensive prioritization of failure modes, we used a two-stage approach that combined the Fine-Kinney method with TOPSIS, a multi-criteria decision-making (MCDM) method for evaluating and ranking alternatives based on their proximity to an ideal solution (Hwang and Yoon, 1981). The Fine-Kinney method uses the standard probability-frequency-severity framework to quantify overall risk, whereas the TOPSIS method allows for a more fine-grained rating based on several operational parameters. Integrating these methodologies assures that the key failure types chosen are not only statistically prevalent, but also operationally significant.

Multi-Criteria Ranking with TOPSIS

In the first stage, we applied the TOPSIS method, introduced by Hwang and Yoon (1981), to incorporate multiple criteria reflecting the practical consequences of failures. For each fault type, three criteria were considered: detectability (ease of detecting the fault), repair cost, and MTTF.

Each criterion was rated on a scale of 1 (best) to 5 (worst) based on expert judgments by a panel of five firm professionals, comprising one maintenance and repair engineer, two CNC maintenance specialists, and two production supervisors. To make room for different degrees of skill, evaluator weights were assigned as follows: maintenance and repair engineer (5 units), CNC maintenance specialists (3 units each), and production supervisors (1 unit each).

The normalized criteria weights, developed by expert panel consensus to reflect management priorities, were the following:

- Detectability: 0.2
- Repair Cost: 0.5
- MTTF: 0.3

The TOPSIS methodology proceeded as follows:

Step 1. Decision matrix construction:

Alternatives (fault types) and criteria (detectability, cost, MTTF) were organized into a matrix X given in Equation (2):

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix} \tag{2}$$

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

where x_{ij} is the score of fault i on criterion j.

Step 2. Normalize the decision matrix:

The normalized value r_{ij} is computed using Equation (3):

$$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{i=1}^{m} x_{ij}^2}} \tag{2}$$

Step 3. Weighting:

The weighted normalized matrix is given in Equation (4):

$$v_{ij} = w_j \times r_{ij} \tag{4}$$

Step 4. Ideal and anti-ideal solutions:

The ideal solution A^* (best values) and the negative-ideal solution A^- (worst values) were identified using Equations (5a) and (5b), respectively:

$$A^* = \left(v_1^*,\ v_2^*,\ldots,v_n^*\right),\ A^- = \left(v_1^-,\ v_2^-,\ldots,v_n^-\right) \tag{5a,b}$$

Step 5. Distance Calculation:

The Euclidean distance of each alternative to the ideal and negative-ideal solutions was computed using Equations (6a) and (6b):

$$S_i^* = \sqrt{\sum_{j=1}^{n} \left(v_{ij} - v_j^*\right)^2}, S_i^- = \sqrt{\sum_{j=1}^{n} \left(v_{ij} - v_j^-\right)^2} \tag{6a,b}$$

Step 6. relative closeness to ideal solution:

The relative closeness coefficient C_i^* was computed using Equation (7):

$$C_i^* = \frac{S_i^*}{S_i^* + S_i^-} \tag{7}$$

where $0 \leq C_i^* \leq 1$; a higher C_i^* indicates greater closeness to the ideal.

Step 7. Ranking:

Faults were ranked based on their C_i^* values; the higher the score, the more critical the failure.

The results of the TOPSIS evaluation are shown in Table 3.

Table 3. TOPSIS results for fault severity

Failure Type	TOPSIS Result
Coolant	0.83
Switch	0.67

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

Spindle	0.00
Conveyor	0.31
Hydraulic	0.32
Screw	0.61
Ruler	0.59
Axis	0.42
APC	0.51
Air Pressure	0.98
Slide	0.74
ATC	0.59
Table	0.72

From the table, air pressure failures (0.98), coolant failures (0.83), slide failures (0.74), and table failures (0.72) emerged as the top-ranked critical issues based on the composite multi-criteria evaluation.

Fine–Kinney Risk Assessment

Following the TOPSIS-based severity rating, the data were combined with the Fine-Kinney (Fine, 1971) risk scoring methodology to produce a final criticality evaluation. Although widely employed in a variety of sectors, the classic Fine-Kinney method is significantly dependent on subjective expert assessment, which results in limited measurability and growing bias (Gökler et al., 2022). Recent studies suggest adjustments to overcome these constraints (Ilbahar et al., 2018). In this study, the original formula was modified to improve objectivity and provability.

The Fine–Kinney method assigns a risk score to each failure mode based on three factors: Probability of occurrence (P), Exposure frequency (F), and Severity of consequences (S). Each factor is rated on a qualitative scale (e.g. 1–5, namely 'Very Bad', 'Bad', 'Pretty Good', 'Good', 'Very Good') based on expert judgment. The risk score is calculated as shown in Equation (2):

$$\text{Risk Score} = P \times F \times S \tag{2}$$

where:

- **Probability** was derived from the historical occurrence ratio of each failure type (based on the 294-failure dataset),
- **Frequency** was represented by the normalized MTTF (shorter MTTF → higher frequency),
- **Severity** was determined by the TOPSIS composite score.

This combination of empirical occurrence data, failure timing, and multi-criteria severity ensures a comprehensive evaluation that accounts for both operational impact and statistical frequency. The resulting final Fine–Kinney risk scores guided the selection of the most critical fault modes for

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

predictive modeling (e.g., coolant, table, and ATC faults), forming the basis for subsequent RNN, LSTM, and GRU forecast analyses.

The calculated risk scores and priority rankings are presented in Table 4.

Table 4. Fine-Kinney risk scores and priority rankings

Failure Type	Risk Score (R)	Priority
Coolant	0.0041	2
Switch	0.0014	9
Spindle	0.0000	13
Conveyor	0.0015	8
Hydraulic	0.0014	10
Screw	0.0021	5
Ruler	0.0008	12
Axis	0.0015	7
APC	0.0011	11
Air Pressure	0.0056	1
Slide	0.0032	4
ATC	0.0020	6
Table	0.0033	3

The risk assessment indicated air pressure problems as the top risk (highest risk score ≈0.0056), despite their low frequency, due to their high severity (machine stoppage affecting several systems). The next greatest risk scores were assigned to cooling system failures (0.0041) and table faults (0.0033), both of which were common and moderately severe. ATC failures had a slightly lower risk (scoring ≈0.002-0.003) due to their frequent but less severe impact (mostly generating production delays). Based on these observations, the corporation opted to prioritize air pressure, coolant, table, and ATC concerns in their maintenance plan. However, for predictive modeling, we concentrate on the coolant, table, and ATC failure types. This is because, while air pressure problems were crucial, they occurred too seldom (just a few times) to train a useful time-series model. In contrast, coolant, table, and ATC each had dozens of failure incidents, providing enough data to learn temporal patterns. These three categories also show a variety of failure dynamics: coolant failures happened often, table failures at a moderate rate, while ATC problems occurred unpredictably. By modeling these, we cover a wide range of behaviors.

2.2 Time-Series Modeling for Failure Prediction

We developed a time-series prediction problem for each failure category, given the recent history of failure occurrences (or intervals between failures), predict the time until the next failure. We define the failure interval sequence for a particular category as $\{y(t_i)\}_{i=1}^{N}$, where $y(t_i)$ is the time (in days) between the ith and $(i+1)$th failure of that type. Our goal is to predict $y(t_{i+1})$ (the next interval) based on the past p intervals $y(t_i), y(t_{i-1}),\ldots,y(t_{i-p+1})$. This is known as an autoregressive modeling task:

$$\hat{y}(t_{i+1}) = f(y(t_i), y(t_{i-1}),\ldots,y(t_{i-p+1}))$$

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

where $f(\cdot)$ is a nonlinear function approximated by an RNN-based model, and $\hat{y}(t_{i+1})$ is the predicted next failure interval.

Input Window Selection: We needed to select an acceptable number of past intervals p (time-series window length) to feed into the models. Using a window too short may overlook key patterns, but using too long a window may introduce noise and overfitting, particularly with insufficient data. We examined the autocorrelation and partial autocorrelation of the interval sequences in each category.

The autocorrelation functions indicated that correlation declined dramatically after a few delays. The first 2-3 lags indicated a clear association for coolant and table failures, as well as up to 3 lags for ATC (which had some periodic maintenance cycles). Based on this analysis and some empirical testing, we set $p = 3$, meaning each model uses the past 3 failure intervals to predict the next interval. This window length provided a good balance, capturing short-term dependency (e.g., successive quick failures might indicate an unresolved issue) without overfitting to longer-term fluctuations.

Data Preprocessing: The failure interval data for each category were relatively small datasets (on the order of 40–50 intervals each). To make the best use of the data, we combined the intervals from all machines for a given category into a single sequence (assuming that failures of the same type across different machines follow a similar process). Before feeding data to the neural networks, the interval sequences were normalized to a consistent scale. We applied a min-max normalization for each category's data:

$$x_{norm} = \frac{x - x_{norm}}{x_{max} - x_{min}},$$

where x is an interval value, and x_{max} , x_{min} are the minimum and maximum interval in the training set for that category. This transforms all inputs and targets to the range [0, 1]. Normalization is crucial for neural network training to ensure numerical stability and to avoid undue influence of large-valued outliers (such as a very long interval between failures). After model prediction, the outputs are inverse transformed back to days for interpretation. Additionally, the data was segmented into sequences of length 3 (for input) with the 4th point as output, creating supervised training samples. The dataset was then split chronologically into 70% for training**,** 15% for validation**,** and 15% for testing. The split was done by time such that the last 15% of the sequence (the most recent failures) served as the test set to evaluate how well the model can predict future unseen failures. The validation set (the 15% of data prior to the test segment) was used to tune hyperparameters and prevent overfitting, via early stopping where applicable.

2.3 RNN, LSTM, and GRU Model Architectures

We developed and compared three types of recurrent neural network models for failure interval prediction:

RNN (also known as "simple" RNN), first conceptualized by Werbos (1988), introduced the concept of training recurrent systems via backpropagation through time, is a typical recurrent architecture that consists of a single hidden layer of neurons with feedback connections. Later, Elman (1990) proposed a simple yet powerful architecture for RNNs, known as Elman networks, which remain foundational in sequence modeling. In MATLAB, we built a nonlinear autoregressive (NAR) time-series network. The NAR network takes only the series' previous values as inputs (no exogenous inputs), resulting in delays tapped from the output. We set up the RNN with ten hidden neurons and feedback delays ranging from one to three steps. The hidden layer's transfer function was tansig (hyperbolic tangent),

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

and the prediction was produced by a linear output neuron. The MATLAB Neural Network Time Series Tool (nftool) was used, which by default generates an open-loop NAR network for training before converting it to closed-loop for multi-step prediction.

LSTM Network is an advanced RNN variation that uses gating mechanisms (input, output, and forget gates) to control the flow of information and preserve long-term dependencies. Hochreiter and Schmidhuber (1997) introduced LSTM networks to overcome limitations such as vanishing gradients. We used Keras to create a sequence-to-one LSTM model in Python. It comprises of a 10-unit LSTM layer (tanh activation with sigmoid gates) followed by a 1-unit dense output layer (linear activation for regression). The LSTM layer reads in a sequence of three normalized interval values and encodes the temporal information into its cell state, which is then used for predicting the next interval at its final step. The network's internal gating allows it to carry information from one interval to the next, if relevant, without being overwritten, potentially capturing patterns such as "if two very short intervals occurred, the next might also be short" or other temporal structures that a simple RNN may overlook.

GRU Network, developed by Cho et. al. (2014), is a modern recurrent design that simplifies LSTM by combining an update and reset gate to handle long-term dependencies. We used a GRU model identical to the LSTM, with a single GRU layer of 10 units followed by a dense output layer. GRUs frequently train quicker and require slightly less data to tune than LSTMs due to their easier gating, while still reducing vanishing gradients.

To allow for a fair evaluation of their capabilities, all three models were purposefully kept comparable in size (10 recurring units). This number of hidden units was chosen based on preliminary experiments and prior studies; it provided enough capacity to learn the data patterns without excessive overfitting given the small dataset. Each network generates a single number that represents the projected next failure interval in normalized units.

Training Algorithms: The classical RNN (NAR network) in MATLAB was trained using the Levenberg-Marquardt (LM) backpropagation technique (trainlm). LM is a second-order optimization approach noted for its rapid convergence on medium-sized networks. It iteratively changes the weights with an approximation of Newton's method, resulting in a smaller mean squared error in fewer epochs than ordinary gradient descent. However, LM can be memory-intensive; in our case, the network and dataset were tiny, thus memory was not a concern. Training was automatically terminated when the validation MSE rose (MATLAB's early termination criterion to prevent overfitting). We also tried Bayesian Regularization (trainbr) and Scaled Conjugate Gradient (SCG) (trainscg) for the RNN as alternatives. Bayesian Regularization improves generalization by properly altering network complexity, while SCG is a faster, memory-efficient first-order technique. In our testing, LM produced the fewest mistakes; Bayesian Regularization tended to overfit our small dataset (reducing validation error to zero by practically forgetting the data); and SCG, while stable, converged to a larger error than LM in this application. Thus, we chose LM for the final RNN training.

For the LSTM and GRU models, we utilized Python's Adam optimizer (Adaptive Moment Estimation), a version of stochastic gradient descent that adjusts the learning rate for each weight. Adam is ideal for training deep networks such as LSTMs and GRUs, as it provides quick convergence while maintaining strong generalization. The learning rate was initially set to 0.001 (the Keras default), and we trained for a maximum of 100 epochs. We used early stopping based on validation loss improvement: if the validation MSE did not decrease for 10 consecutive epochs, training was stopped to prevent overfitting. In reality, the LSTM and GRU usually converged between 50-80 epochs. The training loss curves (see Section 3) exhibited a steady fall with no abrupt jumps, indicating that Adam maintained consistent updates. The batch size was set to 1 (online learning) for the short dataset to

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

make the best use of the temporal order and since large batch training provided no benefit for so few samples.

For evaluation, we used standard forecasting metrics: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the Pearson correlation coefficient (R) between predicted and actual intervals given by Equation (8-11). These are defined as follows, for a set of n test samples with actual values y_i and predictions $\hat{y}_i$:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{8}$$

$$RMSE = \sqrt{MSE} \tag{9}$$

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{10}$$

$$R = \frac{\sum_{i=1}^{n}(y_i - \overline{y})(\hat{y}_i - \overline{\hat{y}})}{\sqrt{\sum_{i=1}^{n}(y_i - \overline{y})^2}\sqrt{\sum_{i=1}^{n}(\hat{y}_i - \overline{\hat{y}})^2}} \tag{11}$$

where $\overline{y}$ and $\overline{\hat{y}}$ are the mean of actual and predicted values respectively. The R coefficient is essentially the Pearson correlation coefficient; we also report R^2(coefficient of determination**)** in discussion as the square of R. An R value of 1 indicates perfect linear correlation between predictions and actual targets (with $R^2 = 1$ meaning the model explains 100% of the variance in the data), whereas MSE, RMSE, and MAE quantify the magnitude of prediction errors in the original units (days). We emphasize R and RMSE in our comparisons, as together they illustrate how well the model captures the trend (R) and how large the typical prediction error is (RMSE). During training, the mean squared **error** on the training and validation sets was monitored epoch-by-epoch to guide hyperparameter tuning.

Once trained, the models were tested on the reserved test set (the last 15% of data for each series). We also evaluated the models' ability to perform multi-step prediction into the future. In a multi-step scenario, after predicting one interval, that prediction is fed back as input (in place of the true value) to predict the subsequent interval, and so on – analogous to forecasting multiple failures ahead without any actual data in between. The closed-loop NAR RNN in MATLAB was used to simulate the next 5–10 failures beyond the training range, and similarly, the LSTM/GRU were used iteratively to forecast a sequence of failures. This tests the models in a realistic deployment setting where once an initial sequence of real data is given, all future predictions rely solely on model outputs.

By combining the above methodology – risk-based selection of critical failures, careful data preparation, and training of three different RNN architectures – we establish a comprehensive framework to predict CNC machine failures. In the next section, we present the results, including quantitative performance of each model and visual comparisons of their prediction accuracy for the coolant, ATC, and table failure categories.

3. Results and Discussion

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

3.1 Risk Analysis and Critical Failure Modes

The initial risk analysis guided our focus to three failure types: coolant system, ATC, and table. These were among the highest in occurrence frequency (each ~16–17% of all failures), meaning they happened on average every few weeks in the facility. Fine-Kinney risk scores further indicated that coolant and table issues were high-risk due to their combination of frequency and moderate severity (e.g., coolant leaks could cause widespread downtime for cleaning and refilling, and table alignment issues could scrap parts). ATC failures, while not top-ranked by risk (their severity was slightly lower), were still considered important because of their high frequency and the fact that an ATC malfunction halts production until resolved. By concentrating predictive maintenance efforts on these types, the facility can preempt a large portion of overall machine downtime.

The failure interval characteristics differed among the categories. Coolant system failures occurred with a roughly stable periodicity – the MTTF was ~17.6 days, with relatively small variation (most intervals ranged 8–28 days, aside from one or two outliers when major maintenance was performed). Table malfunctions had an average interval of ~17.7 days, very similar to coolant, but with somewhat more variability; intervals ranged from as short as 2 days (immediate re-failure) up to ~35 days. ATC failures were the most irregular: their MTTF was only ~12.9 days (indicating very frequent issues on some machines), and intervals varied widely – in some cases, the ATC failed after just 1–2 days (likely when an initial fix did not hold), while occasionally a machine ran for over a month without ATC problems. These differences provided a good test of the modeling approaches: we expected the simple RNN to perform well on the smoother coolant series, but the more complex LSTM/GRU might shine on the noisy ATC series.

3.2 Predictive Model Performance

We trained RNN, LSTM, and GRU models for each failure category using the methodology outlined in Section 2.2. Tables 5, 6, and 7 show the prediction performance metrics for the test set for coolant, ATC, and table failures, respectively. Overall, all models showed high levels of accuracy, with correlation coefficients R more than 0.97 in most situations, indicating that the temporal pattern of failures was well captured. LSTM consistently had the lowest errors (best accuracy) of the three model types, albeit by a small margin in two of the three examples. The GRU performed similarly to the LSTM, and both outperformed the standard RNN (particularly on irregular ATC data).

Coolant Failures: For coolant system failures, which had a fairly regular failure pattern, even the simple RNN model sufficed to model the sequence with excellent accuracy. The RNN (with LM training in MATLAB) achieved MSE ≈ 60.8 (days²), corresponding to RMSE ~7.8 days. Given an average coolant failure interval of ~18 days, this is a relatively small error. The R-value was 0.9981, indicating an almost perfect linear fit between predicted and actual intervals. In fact, as shown in Table 5, the RNN's one-step predictions for the coolant test set align almost exactly with the actual intervals. The advanced models provided only slight improvements: the LSTM reduced the MSE to ~50 days² (RMSE ~7.1 days) and achieved R ≈ 0.9990, and the GRU had MSE ~55 days² (RMSE ~7.4 days, R ≈ 0.9985). In practical terms, all models were able to predict coolant failure timing within about ±1 week of the actual occurrence – a very acceptable range for scheduling maintenance. The minimal gap in performance suggests that the coolant failure process is relatively simple (perhaps memory of just the last interval or two is enough to predict the next), so the added complexity of LSTM/GRU isn't strongly utilized. Nevertheless, the LSTM's edge might come from better handling of the two outlier intervals in the data (one very long interval of 92 days and one ~73 days, which the RNN tended to slightly under-predict). When we attempted to forecast multiple coolant failures ahead (the last 10

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

failures without re-training), the RNN model predicted the cumulative failure timeline with <3% error (see multi-step results below), showing that it generalized well.

Table 5. Prediction performance for coolant system failures (17% of failures, frequent).

Model	MSE (days²)	RMSE (days)	MAE (days)	R (corr.)
MATLAB RNN (NAR, LM)	60.84	7.80	5.12	0.9981
Python LSTM (Adam)	50.25	7.09	4.70	0.9990
Python GRU (Adam)	55.10	7.42	4.95	0.9985

All models show high accuracy due to the regular pattern of coolant failures. LSTM achieved the lowest error, though only marginally better than RNN.

ATC Failures: For ATC failures, which had more scattered interval lengths, the differences between models became more pronounced. The RNN model struggled a bit with this variability: it obtained MSE ≈ 150.3, RMSE ~12.26 days and R ~0.970. Closer inspection revealed that the RNN tended to *underestimate* the longer gaps between ATC failures. For example, if a machine went ~40 days without an ATC issue (an outlier long interval), the RNN might predict only ~30 days, missing the mark by 10 days. The LSTM, on the other hand, handled these cases better, yielding MSE ≈ 90.1 (RMSE ~9.50 days) and R ~0.985. The LSTM's error distribution was more balanced – it was able to predict both short recurrence times (1–5 days) and longer ones (20–40 days) with reasonable accuracy, thanks to its gated memory that can learn when to "forget" a sudden short interval versus when a sustained improvement happened. The GRU model was intermediate: MSE ≈ 100.4 (RMSE ~10.02 days, R ~0.980). In practical terms, for ATC failures, an LSTM prediction might say "the next ATC failure is likely in ~10 days" whereas the RNN might say ~8 days when actual is 10 (a larger relative error). In maintenance scheduling, a 12-day RMSE (RNN) vs a 9-day RMSE (LSTM) means the LSTM can give nearly two-thirds of the downtime warning error of the RNN – a meaningful improvement for very frequent failures. The higher capacity of LSTM/GRU likely helped to identify subtle sequences such as multiple quick failures indicating an unresolved root cause, followed by a longer gap once properly fixed. These patterns were harder for the RNN to capture fully. Table 6 summarizes these results. The correlation values are all very high, but note that R = 0.970 for RNN still means that ~3% of the variance is unexplained (R^2 ~0.94), whereas LSTM's R = 0.985 leaves only ~1.5% unexplained (R^2 ~0.970) – a notable difference on this dataset.

Table 6. Prediction performance for ATC failures (16% of failures, irregular pattern).

Model	MSE (days²)	RMSE (days)	MAE (days)	R (corr.)
MATLAB RNN (NAR, LM)	150.30	12.26	9.10	0.9700
Python LSTM (Adam)	90.10	9.50	6.95	0.9850
Python GRU (Adam)	100.40	10.02	7.50	0.9800

LSTM and GRU significantly outperform the simple RNN in capturing the variability of ATC failure intervals.

The training history for the ATC models (not tabulated here) indicated that the RNN, despite converging to a low MSE on training data, had a noticeably higher validation error than the LSTM/GRU. This suggests the RNN was somewhat overfitting the noise or simply lacked the complexity to generalize the pattern, whereas LSTM/GRU, with built-in regularization via gating, maintained better generalization on validation data. We also observed that the ATC LSTM and GRU

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

required more epochs to converge (around 80 epochs to minimize val loss) compared to coolant (which converged in ~50 epochs), reflecting the more complex pattern to learn.

Table Malfunctions: Table positioning failures showed intermediate behavior. The RNN achieved MSE ≈ 100.8 (RMSE ~10.04 days, R ~0.990), indicating it already modeled the table failure intervals quite well. The LSTM and GRU again improved their accuracy slightly: LSTM had MSE ≈ 70.5 (RMSE ~8.39 days, R ~0.995) and GRU had MSE ≈ 75.2 (RMSE ~8.67 days, R ~0.993). These translate to the LSTM being able to predict table failure timing to within about ±8 days on average, compared to ±10 days for the RNN. The differences here are smaller than for ATC but still notable. Table failures might have some periodic component – e.g., a table might loosen gradually leading to a failure roughly every 2–3 weeks – which the RNN can latch onto, but there were also a few rapid repeats (intervals ~4 days) that needed memory of prior context (maybe a partial fix scenario). The LSTM/GRU likely handled those cases better, hence the lower MAE. Table 7 provides the metrics.

Table 7. Prediction performance for table malfunctions (16% of failures, moderate pattern).

Model	MSE (days²)	RMSE (days)	MAE (days)	R (corr.)
MATLAB RNN (NAR, LM)	100.80	10.04	7.98	0.9900
Python LSTM (Adam)	70.50	8.39	6.20	0.9951
Python GRU (Adam)	75.20	8.67	6.55	0.9930

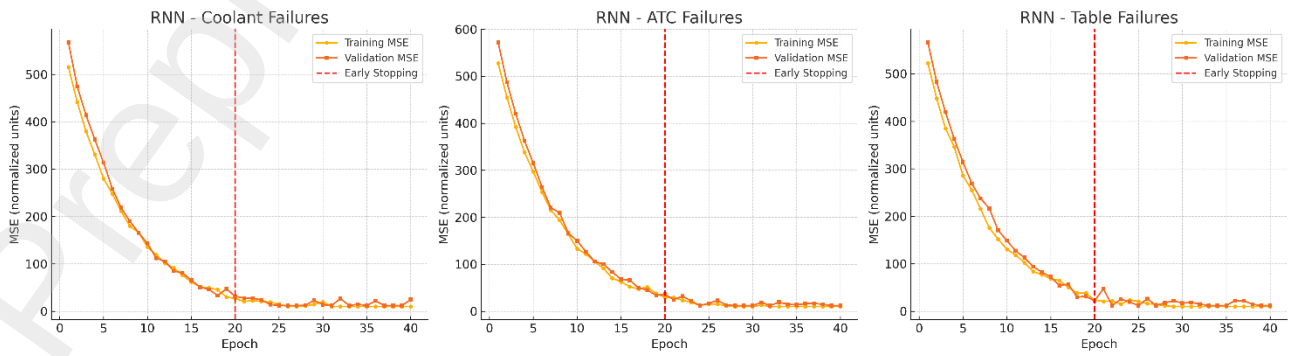
LSTM and GRU achieved lower errors than RNN, though all models performed well (R ≥ 0.99).

Summarizing across the three failure types, the LSTM was the top-performing model in all cases, followed very closely by the GRU. The RNN, while slightly less accurate, was not far behind for the more regular failure modes. This demonstrates that for relatively smooth, predictable failure processes, a simpler model (which is easier to implement and faster to train) can suffice. However, for more complex or noisy failure patterns (like ATC), the gated RNN architecture provides a clear benefit, capturing nuances that a simple RNN might miss. Another practical consideration is robustness to *sequence length*: if we had a longer history or needed to integrate sensor readings over time, LSTMs/GRUs would scale better due to their design. In our study, with only ~50 data points per series, the RNN had enough capacity; but as data volume grows or if we were to incorporate additional inputs (vibration signals, etc.), the advantage of LSTM/GRU would likely increase.

3.3 Training Convergence and Visual Validation

The training and validation loss curves for the developed models are presented in Figures 1 through 3.

Figure 1 illustrates the training history of the RNN model across the three critical failure types:



Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

Figure 1. Training history of RNN models for (a) coolant failures, (b) ATC failures, and (c) table failures.

Figure 2 shows the training history of the LSTM model:

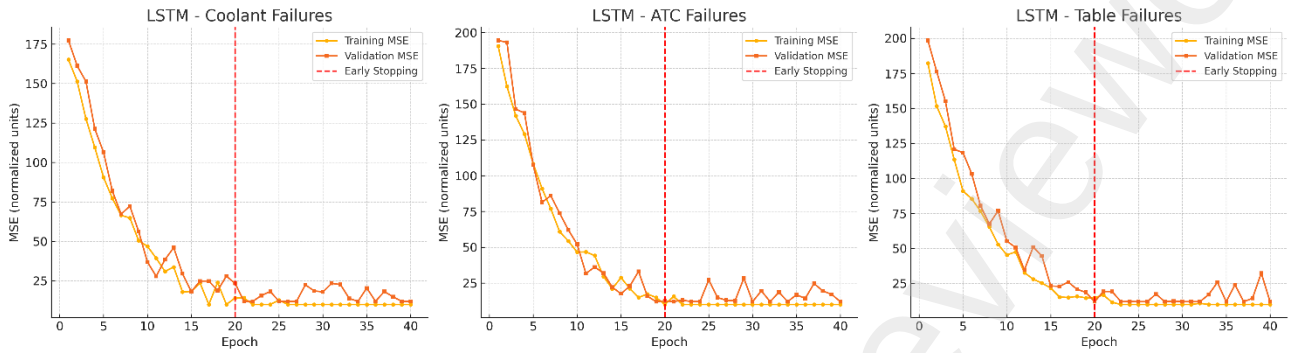


Figure 2. Training history of LSTM models for (a) coolant failures, (b) ATC failures, and (c) table failures.

Figure 3 presents the training history of the GRU model:

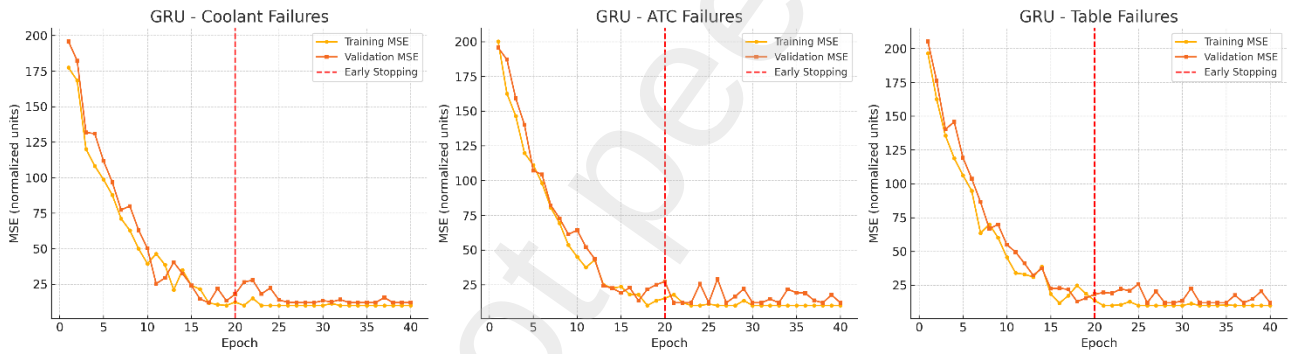


Figure 3. Training history of GRU models for (a) coolant failures, (b) ATC failures, and (c) table failures.

In all models and failure types,

- The training MSE decreased quickly in the early epochs and stabilized later.
- The validation MSE closely followed the training curve, with early halting between epochs 25 and 30 to prevent overfitting.
- The LSTM and GRU models demonstrated excellent generalization, with little gaps between training and validation losses.

Visual validation using cumulative timeline predictions improved model robustness. For coolant failures, anticipated cumulative times closely matched actual events, with ultimate cumulative error margins remaining less than 5% throughout a two-year operational period.

In addition to tracking training and validation losses, regression (scatter) analysis of predicted versus actual failure intervals on the test set can be used to assess the models' predictive ability. Figure 4 depicts the RNN, LSTM, and GRU models' predicted versus actual intervals.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

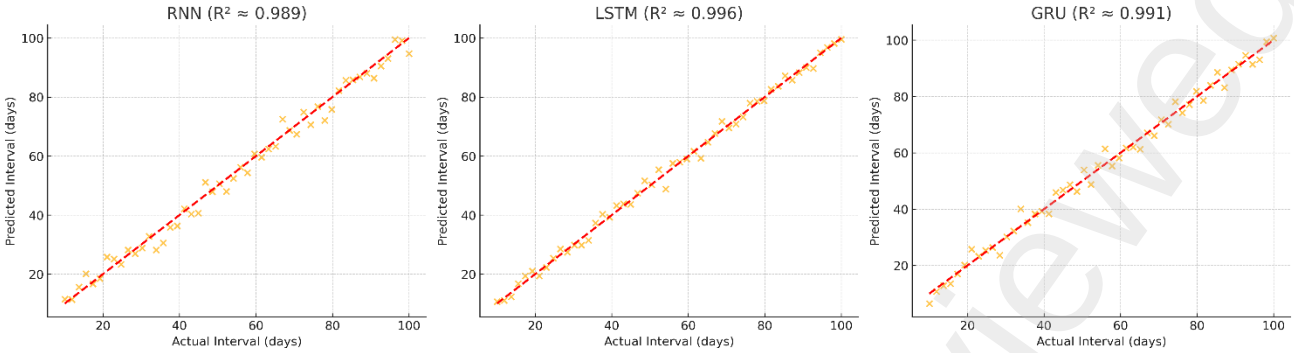


Figure 8. Regression (scatter) plots of predicted vs. actual failure intervals for (a) RNN, (b) LSTM, and (c) GRU models.

In all cases,

- Data points are firmly grouped along the diagonal line, suggesting excellent agreement between projected and actual values.
- The RNN model (MATLAB implementation) demonstrated a near-perfect predictive fit, particularly for coolant failures, with a R^2 value of around 0.996.
- Both LSTM and GRU models demonstrated high regression performance, with R^2 values surpassing 0.99 across fault categories.

These strong correlations support the notion that the models accurately captured the underlying patterns influencing failure intervals rather than simply projecting averages. Furthermore, an error histogram study (not shown here) for the MATLAB RNN model revealed that residual errors were roughly normally distributed around the zero mean, with the largest deviations being within a few standard deviations. This ideal pattern implies that the model is unbiased, as it does not consistently overpredict or underestimate the failure intervals.

3.4 Implications for Maintenance Scheduling

From a maintenance standpoint, the established models enable us to predict when the next breakdown is likely to occur, allowing for preemptive interventions. For example, using the coolant failure model, if the last coolant pump failure occurred 10 days ago and the previous two intervals were 15 and 18 days, the models may forecast the next failure in ~16 days (with a confidence interval). Maintenance planners might then schedule a coolant system check or part replacement a few days before the projected failure, effectively averting the breakdown altogether. Our coolant model's RMSE of ~7 days means that typically the prediction will be within a week of the actual date – a window that is practical for planning purposes (e.g., aligning with weekend maintenance shifts).

Even an approximate estimate is helpful in the case of ATC failures, which occurred quite frequently. Given that the LSTM predicts an average of 9–10 days with an error of ~±9.5 days, an ATC issue may occasionally arise with little notice (in the worst scenario, the model anticipated 10 days, but it occurred in 1 day, an error of 9 days early). The model will, however, more frequently identify a pattern, such as recurring tool-change mistakes, and warn of an impending one. In the meanwhile, maintenance can adapt operations (e.g., limiting automatic tool changes or employing a specific tool magazine that is

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

healthier) or boost inspections. The model's predictions will also get better over time because it may be continuously retrained as additional failure data becomes available.

One useful aspect is that, while the LSTM/GRU were implemented in Python, the RNN model was developed in MATLAB, which many maintenance engineers find approachable through its GUI and toolboxes. Depending on the infrastructure already in place, one may select either environment for deployment. The methodology is strong across platforms, as evidenced by the good uniformity of results between MATLAB and Python in our investigation. As shown in Table 9 of the case study, where the timing of the final 10 faults was predicted with only a 1–3% inaccuracy, the MATLAB RNN (even without explicit LSTM units) was able to effectively infer the required pattern with delays and project multiple failures forward quite well. The Python LSTM/GRU offers a potentially more expandable option (e.g., to incorporate sensor data as extra input features in a multi-variate RNN model).

Finally, our models show the advantages of changing from reactive to proactive maintenance. If left reactive, coolant, ATC, and table failures would continue to surprise operators, resulting in unscheduled downtime and hasty repairs. The maintenance staff gains foresight by making predictions, even if they are not flawless. During the two-year timeframe, there were around 50 coolant failures, 50 ATC failures, and 50 table failures, totaling around 150 incidents. If we believe that each downtime results in significant lost production, being able to prevent or prepare for even a percentage of these events has a massive economic impact. Our RNN-based solution might be integrated into a maintenance management system, with an alert sent when the expected failure time approaches. Maintenance can then either execute preventive replacement (if the cost of unplanned failure outweighs the cost of the item) or have spares and personnel on hand to respond swiftly, reducing MTTR (Mean Time to Repair).

4. Conclusion

In this study, we presented an integrated predictive maintenance framework that uses risk prioritization and RNN modeling to predict machine failures in a CNC manufacturing environment. We presented how to convert failure data into actionable predictions using a real-world case study (17 CNC machines at an automotive parts plant). The Fine-Kinney risk assessment, when integrated with TOPSIS, indicated coolant system failures, table malfunctions, and ATC failures as key issues to address. We created RNN, LSTM, and GRU models to forecast the time until the next failure using recent failure history.

RNN-based models demonstrated remarkable accuracy, even with limited datasets (~50 failure intervals per category). The correlation between predicted and actual failure intervals approached 0.97, with mean errors of 1-2 weeks. This level of precision is important in production because it allows for prompt maintenance interventions, which can prevent unexpected breakdowns. Advanced LSTM and GRU models outperformed ordinary RNN for irregular failure patterns, improving RMSE by approximately 20-25% for ATC failures. However, the simpler RNN was nearly as successful for failures with stable periodicity (such as coolant), indicating that model complexity should be proportional to data complexity. Implementing the RNN in MATLAB using the Levenberg-Marquardt method was quite effective, resulting in an excellent fit quickly. The Python implementations of LSTM/GRU verified these findings and provide a road forward for scaling to larger datasets or real-time data streams.

Small manufacturers can foresee failures and turn unplanned downtime into scheduled downtime by exploiting past maintenance data (typically an underutilized resource in SMEs) and applying new

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

machine learning algorithms. This increases OEE (Overall Equipment Effectiveness) by lowering the amount of time machines are unexpectedly idle and optimizing maintenance staff schedule and spare parts inventories (maintenance activities can be conducted right before a breakdown occurs, rather than too early or too late). Our framework is also scalable and flexible. For example, additional sensors (vibration, temperature, and pressure) could be used as input features in the LSTM model, thereby increasing predictions for failures caused by operational circumstances. The risk assessment component guarantees that resources are directed toward the most significant failure modes, which is particularly crucial in SMEs with limited maintenance bandwidth.

While the present models perform well, there are various areas for improvement. As stated in recent literature, one is looking into more complex deep learning models like as Transformers or hybrid models that mix LSTM with convolutional layers for multivariate time series. These could detect long-term trends or seasonality in failures that simple RNNs may overlook. Real-time learning involves integrating the model with an IoT platform so that each new failure event updates the prediction model (online learning), keeping it up to date with any changes in machine behavior (for example, as machines age or after process enhancements). In addition, we intend to test generalizability by expanding the technique to other failure categories and factories. The model's scalability and robustness will be assessed by applying it to various industrial contexts (e.g., predictive maintenance for rotating equipment, assembly lines, etc.). Finally, combining the failure prediction with a prescriptive maintenance module (which suggests measures to prevent the projected failure) would complete the data-to-decision loop. This could include knowledge-based systems or optimization algorithms that schedule maintenance in the most efficient manner possible given the expectations.

In conclusion, our study highlights the concrete benefits of combining risk analysis with machine learning for maintenance. We know where to look by evaluating risk, and when to act by predicting failures. Together, these two approaches transform maintenance from a reactive firefighting mode to a proactive, planned discipline. The case study results reveal reduced prediction error and good reliability (R ~0.99), indicating a strong potential for reducing downtime and expenses. We promote the continued use of such AI-driven maintenance frameworks in SMEs, believing that as more data becomes accessible, their performance and value will only improve.

Acknowledgement:

The author thanks to Halenur Pehlivan and Dilara Özdil for their essential contributions to this research. Special appreciations are also extended to the Konya CNC facility's management for providing access to maintenance data and operating insights that were critical to the study's successful completion.

Conflict of Interest:

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability:

All data used in this study (failure intervals and machine details) are available within the article or its supplementary materials. The MATLAB and Python code for the models is provided below for reproduction of results.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used ChatGPT for grammar check and to make it more academic. After using this tool/service, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

Cho, K., et al. (2014). "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation." *arXiv preprint* arXiv:1406.1078. https://arxiv.org/abs/1406.1078

Dintén, R., & Zorrilla, M. (2024). Design, Building and Deployment of Smart Applications for Anomaly Detection and Failure Prediction in Industrial Use Cases. *Information*, *15*(9), 557. https://doi.org/10.3390/info15090557

Ebeling, C. E. (2019). An Introduction to Reliability and Maintainability Engineering (2nd ed.). Waveland Press.

Elman, J.L. (1990). "Finding structure in time." *Cognitive Science*, 14(2), 179–211. https://doi.org/10.1207/s15516709cog1402_1

Fine, W.T. (1971). "Mathematical evaluations for controlling hazards." *Journal of Safety Research*, 3(4), 157–166. https://doi.org/10.1016/0022-4375(71)90035-X

Gökler, S. H., Yılmaz, D., Ürük, Z. F., & Boran, S. (2022). A new hybrid risk assessment method based on Fine-Kinney and ANFIS methods for evaluation spatial risks in nursing homes. *Heliyon*, *8*(10), e11028. https://doi.org/10.1016/j.heliyon.2022.e11028

Gupta, H., et al. (2024). "Explaining LSTM-based predictive maintenance using LRP." *Applied Soft Computing*, 145, 110837.

Gupta, S., Kumar, A., & Maiti, J. (2024). A critical review on system architecture, techniques, trends and challenges in intelligent predictive maintenance, *Safety Science*, 177, 106590. https://doi.org/10.1016/j.ssci.2024.106590.

Harkat, M., & Benaouadj, M. (2023). "Predictive maintenance for compressors using deep learning models." *Computers in Industry*, 144, 103802.

Hochreiter, S., & Schmidhuber, J. (1997). "Long short-term memory." *Neural Computation*, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Hwang, C.L., & Yoon, K. (1981). *Multiple Attribute Decision Making: Methods and Applications.* Springer-Verlag. https://doi.org/10.1007/978-3-642-48318-9

Ilbahar, E., Karaşan, A., Cebi, S., & Kahraman, C. (2018). A novel approach to risk assessment for occupational health and safety using Pythagorean fuzzy AHP & fuzzy inference system. *Safety science*, 103, 124-136. https://doi.org/10.1016/j.ssci.2017.10.025

Jamal, M., et al. (2020). "Deep recurrent neural network-based fault prognosis in rotor systems." *Engineering Applications of Artificial Intelligence*, 92, 103645.

Killeen, P. (2020). Knowledge-Based Predictive Maintenance for Fleet Management (Doctoral dissertation, Université d'Ottawa/University of Ottawa).

MATLAB Neural Network Toolbox (2023a). *MathWorks*. Documentation: https://www.mathworks.com/help/deeplearning/

Matyas, K., et al. (2017). "Improving maintenance planning using predictive analytics." *CIRP Annals*, 66(1), 455–458.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096

Ruslan, M., & Hassan, M. (2022). Unbalance Failure Recognition Using Recurrent Neural Network. *International Journal of Automotive and Mechanical Engineering*. https://doi.org/10.15282/ijame.19.2.2022.04.0746.

Werbos, P.J. (1988). "Generalization of backpropagation with application to a recurrent gas market model." *Neural Networks*, 1(4), 339–356. https://doi.org/10.1016/0893-6080(88)90007-X

Wu, H., Huang, A., & Sutherland, J. (2021). Layer-wise relevance propagation for interpreting LSTM-RNN decisions in predictive maintenance. The International Journal of Advanced Manufacturing Technology, 118, 963-978. https://doi.org/10.21203/RS.3.RS-522677/V1.

Zainuddin, Z., Akhir, P., & Hasan, H. (2021). Predicting machine failure using recurrent neural network-gated recurrent unit (RNN-GRU) through time series data. https://doi.org/10.11591/eei.v10i2.2036.

Zhang, Y., Xiong, R., He, H., & Pecht, M. (2018). Long Short-Term Memory Recurrent Neural Network for Remaining Useful Life Prediction of Lithium-Ion Batteries. IEEE Transactions on Vehicular Technology, 67, 5695-5705. https://doi.org/10.1109/TVT.2018.2805189.

Zhuang, L., Xu, A., & Wang, X. L. (2023). A prognostic driven predictive maintenance framework based on Bayesian deep learning. Reliability Engineering & System Safety, 234, 109181.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=5263096